Lesson 6: The Full RAG Web Application & Checkpoint Project (Day 33)

> ***Goal:*** *Today is the culmination of the entire course curriculum! We combine everything learned into a* ***complete, interactive web application*** *using* ***Streamlit*** *and our* ***RAG v2 backend*** *for a real Lahore tea shop:* ***Chai Wala****. We then add advanced production upgrades:* ***PII Masking (anonymizing CNICs & phone numbers)*** *and* ***Two-Stage Re-Ranking (retrieve 6, keep the best 3)****.*

☕ The Capstone Project: Chai Wala Support Bot

We build a full graphical web chat interface for **Chai Wala** (a tea shop in Gulberg, Lahore). Customers can ask about:

- The menu (Chai varieties, parathas, samosas, prices)
- Delivery policies & minimum order thresholds
- Timings and payment methods (JazzCash, Nayapay, Cash on Delivery)
- Refund and replacement policies

□ Vocabulary: Terms You Must Know

Word	What it Means	Everyday Real-World Analogy
Streamlit	A fast Python framework that turns data scripts into interactive web applications without needing HTML, CSS, or JavaScript.	A prefabricated booth you can set up in 5 minutes without building brick walls.
`st.session_state`	Streamlit's internal memory dictionary that persists data (like chat history) across browser page refreshes.	A whiteboard in the room that doesn't get erased when someone walks through the door.
PII (Personally Identifiable Information)	Any sensitive personal data that points to a specific human: CNIC numbers, phone numbers, emails, credit cards.	A person's fingerprint or passport number.
Masking (Redaction)	Replacing sensitive personal data with a generic label (e.g., `35202-1234567-1` ➔ `[CNIC]`).	Taking a black marker and crossing out sensitive text on a photocopied form.
Swap & Restore (Pseudonymization)	Temporarily replacing private info with a tag (`<PHONE_1>`), letting the LLM process the prompt, and swapping the real number back before showing the user.	Giving a customer a numbered coat-check token, and returning their real coat when they leave.
Re-Ranking (Two-Stage Retrieval)	Asking Chroma for **6 chunks** (fast & rough), then using a more precise second scoring pass to pick the **top 3** for the LLM.	An HR recruiter shortlisting 6 CVs quickly, then the department manager carefully reading all 6 to interview the top 3.

□ Architecture of the Full Application

```
========================= FRONTEND: STREAMLIT ========================
User enters chat message ⟶ st.chat_input("Ask Chai Wala...")
                            Display in chat bubbles: st.chat_message()
                                        |
                                        ▼
==================== MIDDLEWARE: SECURITY & PII ====================
1. MASK / HIDE PII: Replace CNIC/Phone numbers with tags
2. INPUT GUARDRAIL: Block prompt injections
                                        |
                                        ▼
====================== BACKEND: RAG PIPELINE ========================
1. RETRIEVE: Fetch top 6 chunks from ChromaDB ("chai_db")
2. RE-RANK: Keyword overlap / Cross-Encoder bubbles the best to Rank 1
3. CUTOFF: Check if top chunk distance < 0.85
4. AUGMENT: Inject top 3 chunks with source tags: [menu.txt]
5. GENERATE: LLM writes friendly, 3-sentence grounded response
                                        |
                                        ▼
======================== DISPLAY TO CUSTOMER ========================
Browser displays answer with citation: "A cup of Karak Chai is Rs 120. [menu.txt]"
```

Part 1: The Streamlit Web App Code (`chai_wala_rag_app.py`)

Here is the complete source code of the application:

```
import os
from pathlib import Path
import streamlit as st
import chromadb
from dotenv import load_dotenv
from openai import OpenAI

# 1. SETUP & CLIENTS
load_dotenv()
client = OpenAI(
    base_url="https://openrouter.ai/api/v1",
    api_key=os.getenv("OPEN_ROUTER_KEY")
)
MODEL = "nvidia/nemotron-3.5-lightning:free"
EMB_MODEL = "nvidia/nemotron-3-embed-1b:free"

def ask(messages, temperature=0):
    try:
        r = client.chat.completions.create(model=MODEL, messages=messages, temperature=temperature)
        return (r.choices[0].message.content or "").strip()
    except Exception as e:
        return f"ERROR: {e}"

def embed(texts):
    r = client.embeddings.create(model=EMB_MODEL, input=texts)
    return [item.embedding for item in r.data]

# 2. DOCUMENT INGESTION & CHUNKING
def load_documents(folder):
    docs = []
    for path in sorted(Path(folder).glob("*.txt")):
        docs.append({"source": path.name, "text": path.read_text(encoding="utf-8")})
    return docs

def chunk_by_paragraph(text):
    parts = [p.strip() for p in text.split("\n\n")]
    return [p for p in parts if len(p) > 30]

def chunk_documents(docs):
    chunks = []
    for doc in docs:
        for i, text in enumerate(chunk_by_paragraph(doc["text"])):
            chunks.append({"id": f"{doc['source']}-{i}", "source": doc["source"], "text": text})
    return chunks

chunks = chunk_documents(load_documents("chai_wala_docs"))

# 3. VECTOR DATABASE (CHROMADB)
db = chromadb.PersistentClient(path="chai_db")
shop = db.get_or_create_collection(
    name="chai_wala_docs",
    metadata={"hnsw:space": "cosine"}
)

# Embed only once to save daily quota
if shop.count() == 0:
    shop.add(
        ids=[c["id"] for c in chunks],
        documents=[c["text"] for c in chunks],
        metadatas=[{"source": c["source"]} for c in chunks],
        embeddings=embed([c["text"] for c in chunks])
    )

# 4. RETRIEVAL & RAG LOGIC
def retrieve(question, n=3):
```

```
    r = shop.query(query_embeddings=embed([question]), n_results=n)
    return [
        {"text": t, "source": m["source"], "distance": d}
        for t, m, d in zip(r["documents"][0], r["metadatas"][0], r["distances"][0])
    ]

I_DONT_KNOW = "Sorry, I don't know. Please call 042-1234567."
CUTOFF = 0.85

RAG_SYSTEM = f"""You are the support bot of Chai Wala, a tea shop in Gulberg, Lahore.
Rules:
- Answer using ONLY the context. Never make up a price, place or time.
- If the answer is not in the context, reply exactly: "{I_DONT_KNOW}"
- At the end, write the source file in brackets, like [menu.txt].
- Be friendly. Reply in at most 3 short sentences."""

def rag(question):
    found = retrieve(question)
    if found[0]["distance"] > CUTOFF:
        return I_DONT_KNOW

    context = "\n\n".join(f"[{f['source']}] {f['text']}" for f in found)
    messages = [
        {"role": "system", "content": RAG_SYSTEM},
        {"role": "user", "content": f"Context:\n{context}\n\nQuestion: {question}"}
    ]
    return ask(messages)

# 5. STREAMLIT WEB UI
st.title("☕ Chai Wala Support Bot")
st.caption("Answers come from verified Chai Wala documents. Sources shown in [brackets].")

# Initialize chat session history
if "messages" not in st.session_state:
    st.session_state.messages = [
        {"role": "assistant", "content": "Assalam o Alaikum! Ask me about our menu, delivery, timings or refunds. Call 042-1234567 f
    ]

# Display conversation bubbles
for m in st.session_state.messages:
    st.chat_message(m["role"]).write(m["content"])

# User chat input bar
question = st.chat_input("Ask Chai Wala...")
if question:
    # 1. Show user message
    st.chat_message("user").write(question)
    st.session_state.messages.append({"role": "user", "content": question})

    # 2. Generate RAG reply
    reply = rag(question)

    # 3. Show assistant reply
    st.chat_message("assistant").write(reply)
    st.session_state.messages.append({"role": "assistant", "content": reply})
```

□ How to Run It:

Open your PowerShell terminal inside the project directory and execute:

```
streamlit run "notebooks/Week 7/Day 33/chai_wala_rag_app.py"
```

Streamlit will automatically open your default web browser to `http://localhost:8501`.

Part 2: Peer Testing (The 8 Question Types)

A software application is never tested only on questions you already know the answer to!

Professional QA testing requires testing **8 distinct query types**:

#	Question Category	Example Question for Chai Wala	Expected Behavior
1	**Direct Fact**	*"How much is a samosa?"*	Gives exact price with citation `[menu.txt]`.
2	**Synonyms / Different Words**	*"What does it cost to bring chai to my house?"*	Matches delivery policy `[delivery.txt]`.
3	**Explicit "No" in Docs**	*"Do you have chicken samosa?"*	States only aloo/keema available `[menu.txt]`.
4	**Multi-Fact Synthesis**	*"Are you open at 1:30 pm on Friday?"*	Synthesizes Friday prayer break timings.
5	**Calculation / Math**	*"10 cups of Kashmiri chai: is delivery free?"*	Calculates total > Rs 1000 and states delivery is free.
6	**Spelling Mistakes**	*"dilivery charges?"*	Embeddings handle typos naturally.
7	**Roman Urdu**	*"Refund kaise milega?"*	Answers refund policy accurately.
8	**Out of Bounds / Unrelated**	*"Is there car parking?"* / *"Who won the cricket match?"*	Triggers `CUTOFF` ➔ *"Sorry, I don't know. Please call 042-1234567."*

Part 3: Protecting Sensitive PII Data

Why PII Masking is Legally Critical

Under privacy laws (and good engineering ethics), users will inevitably type private data into customer support chats:

"My CNIC is 35202-1234567-1 and my phone is 0300-1234567. Where is my order?"

If you send that prompt to OpenRouter / external cloud APIs:

1. That private citizen's ID is stored in cloud server logs.
2. It is saved in your internal analytics databases.

Method 1: Masking (Redaction)

```
import re

PII_PATTERNS = {
    "CNIC":  r"(?<!\d)\d{5}-?\d{7}-?\d(?!\\d)",            # e.g. 35202-1234567-1
    "PHONE": r"(?<!\d)(?:\+92|0)3\d{2}-?\d{7}(?!\d)",      # e.g. 0300-1234567
    "EMAIL": r"[\w.+-]+@[\w-]+\.[\w.-]+",
    "CARD":  r"(?<!\d)(?:\d[ -]?){13,16}(?!\d)"
}

def mask_pii(text):
    for name, pattern in PII_PATTERNS.items():
        text = re.sub(pattern, f"[{name}]", text)
    return text

raw = "My CNIC is 35202-1234567-1 and phone is 0300-1234567. Order status?"
print(mask_pii(raw))
# Output: "My CNIC is [CNIC] and phone is [PHONE]. Order status?"
```

The LLM can still answer the order question, but the customer's real CNIC and phone never leave your local machine!

Method 2: Swap & Restore (Pseudonymization)

What if the bot needs to confirm: *"Thanks! We will call you on 0300-1234567"*?

We swap the number with a temporary tag, send the tag to the LLM, and swap the real number back before the user sees it:

```
def hide_pii(text):
    secrets = {}
    for name, pattern in PII_PATTERNS.items():
        for i, val in enumerate(re.findall(pattern, text), 1):
            tag = f"{name}_{i}"
            secrets[tag] = val
            text = text.replace(val, f"<{tag}>")
    return text, secrets

def restore_pii(text, secrets):
    for tag, val in secrets.items():
        text = text.replace(f"<{tag}>", val)
    return text

# Demonstration:
safe_prompt, vault = hide_pii("Please call me on 0300-1234567 regarding my chai.")
print("Sent to cloud LLM:", safe_prompt)  # "... call me on <PHONE_1> ..."
print("Kept on local machine:", vault)     # {'PHONE_1': '0300-1234567'}

llm_reply = "We received your request and will call <PHONE_1> shortly."
final_user_view = restore_pii(llm_reply, vault)
print("Displayed to user:", final_user_view)
# Output: "We received your request and will call 0300-1234567 shortly."
```

Part 4: Two-Stage Re-Ranking

The Problem: Cosine Similarity is a Coarse Filter

Suppose a customer asks:

"Do I get extra money if I work after 5 pm?"

When Chroma searches 15 chunks, the true relevant chunk (*"Work after 5 pm is overtime..."*) might rank **5th** because general office hours chunks happen to share similar words.

If you only retrieve `n=3`, the overtime chunk is **permanently lost**!

The Solution: Two-Stage Retrieval

1. **Stage 1 (Coarse Retrieval):** Fetch **6 chunks** (`n=6`) from Chroma.
2. **Stage 2 (Fine Re-Ranking):** Run a second-pass ranking function to score and re-order the 6 chunks, then pass only the **top 3** to the LLM.

```
STOP_WORDS = {
    "do", "i", "get", "if", "the", "a", "an", "is", "are",
    "my", "me", "to", "of", "for", "in", "on", "and", "what"
}

def clean_words(text):
    return {w for w in re.findall(r"[a-z0-9]+", text.lower()) if w not in STOP_WORDS}

def rerank(question, candidate_chunks, top=3):
    q_words = clean_words(question)

    # Sort candidate chunks by how many significant question keywords they contain:
    ranked = sorted(
        candidate_chunks,
        key=lambda chunk: len(q_words & clean_words(chunk["text"])),
        reverse=True
    )
    return ranked[:top]

# In your RAG pipeline, update retrieval:
def advanced_rag(question):
    # Step 1: Broad search for 6 candidates
    candidates = retrieve(question, n=6)

    # Step 2: Re-rank and keep only the sharpest 3
    top_chunks = rerank(question, candidates, top=3)

    # ... proceed to augment and generate with top_chunks ...
```

Part 5: Weekly Skill Checkpoint Exam & Answers

Here are the 8 fundamental evaluation questions tested at this checkpoint, along with the ideal software engineer answers:

1. What do the three letters of RAG stand for?

- **R (Retrieve):** Vector database searches and returns relevant text chunks based on semantic similarity.
- **A (Augment):** Python injects the retrieved chunks into the prompt as a grounded context block.
- **G (Generate):** The LLM synthesizes the final answer using only the provided context.

2. Why do we chunk documents instead of passing the whole file?

- Passing whole files blurs embedding vectors (the "smoothie" problem), exceeds token limits, and costs unnecessary money. Making chunks too small strips out critical context. Chunks must represent **one complete standalone idea**.

3. What does Chroma store for each chunk? Why save the source?

- Chroma stores: (1) `id` (unique key), (2) `documents` (raw text), (3) `metadatas` (source filename, permissions), and (4) `embeddings` (vector coordinates). The source is saved to provide **traceable citations** back to verified policy files.

4. Why use `PersistentClient` and `if store.count() == 0`?

- `PersistentClient` saves database collections to disk (`company_db`). Checking `count() == 0` ensures chunks are embedded **only once**, preventing quota exhaustion and avoiding re-embedding identical files.

5. When a RAG bot gives a wrong answer, what do you look at first?

- **Always inspect the retrieved chunks first.** If the correct chunk was never retrieved, your retrieval broke (fix chunking, overlap, or search parameters). If the correct chunk was retrieved but the bot hallucinated, your generation prompt broke.

6. What is PII, and how do you protect it?

- PII is Personally Identifiable Information (CNIC, phone, credit card). You protect it by **masking/redaction** (replacing with `[CNIC]`) or **pseudonymization / swap & restore** (swapping with `<TAG>`, calling LLM, and restoring original values locally).

7. What does re-ranking do? Why retrieve more chunks first?

- Re-ranking is a two-stage search strategy. Cosine similarity in vector space is fast but coarse; high-value chunks can get pushed down to rank 4 or 5. Retrieving 6 chunks captures them, and re-ranking promotes the best chunk to rank 1.

8. Your manager says: "Let's fine-tune a model on our menu!" What do you answer?

- *"No. Menu items, prices, and seasonal specials change constantly. Fine-tuning is expensive, takes hours to train, and cannot cite sources. We should use RAG for menu facts because updating a text file in Chroma takes 2 minutes and costs nothing."*